Take a look at the following:

```
function getQuoteBot ()
{
    $quotes = [
        "We've gone on holiday by mistake.",
        "We're bona fide, we're not from London.",
        "We want the finest wines available to humanity.",
    ];

    $i = rand(0, 2);

    return function () use ($quotes, $i): string {
        return $quotes[$i] . PHP_EOL;
    };
}

$bot = getQuoteBot();
print $bot(); // Outputs one of the strings in the $quotes array
```

The parent function `getQuoteBot()` contains an array and generates a random number between 0 and 2. These are explicitly passed into the anonymous function, which is the return value. Obviously, this could be cleaner code; the thing to focus on is the anonymous function. It only contains a single expression, which is returned to the caller. An ideal candidate for conversion to an arrow function.

Here's what the anonymous function looks like when rewritten as an arrow function:

```
return fn (): string => $quotes[$i] . PHP_EOL;
```

The main thing to take away here is that we have access to the parent function's scope (no need to explicitly make them available with `use`) and we do not need to use a `return` keyword. The arrow function automatically returns the value of its single expression (in this case, one of the strings in `$quotes` concatenated with a `PHP_EOL` constant). An altogether cleaner and easier-to-read syntax.